Production-Style
Ecommerce Analytics Warehouse

End-to-End Project Strategy

Stack: SQL • BigQuery • dbt • Apache Airflow • Great Expectations • Looker Studio / Power BI

Stakeholders & Business Owners

Stakeholder	Key Metrics & Concerns
CEO	Revenue, orders, AOV, growth trends, cancellations
Operations	Delivery performance, late orders, seller SLA compliance
Product / Category Managers	Top categories, basket mix, repeat customer behaviour
Customer Team	Cohort analysis, repeat rate, review score trends
Marketplace Team	Seller performance, seller concentration, geographic coverage

Datasets

Primary Dataset — Olist

The Olist dataset is the main source of truth for this project, covering orders, sellers, payments, reviews, and logistics data.

Olist Coverage

✓ Strong: orders, sellers, payments, reviews, logistics

✕ Weak / missing: ad attribution, CAC, true product margin

Optional Enrichment Sources

- GA4 sample ecommerce dataset — clickstream and funnel analysis. Label clearly as obfuscated sample data, not the primary truth source.
- Synthetic COGS table (self-created) — enables margin and profit analysis if deeper financial insight is needed.

Phase 1 — Business Scope & Repo Setup

Define 10–15 Business Questions Before Writing SQL

Anchor every model to a real stakeholder question. Example questions:

- What drives revenue by month, category, seller, and state?
- Which sellers most often miss delivery expectations?
- Which customer cohorts repeat the most?
- Which payment methods are associated with higher cancellation rates?
- How does review score vary by delivery delay and distance?

Repository Structure

Use Docker Compose for a clean local environment (dbt, Airflow, helper services) and GitHub Actions for CI on every push or pull request.

Directory / File	Purpose
external/	Third-party or external reference files
ingestion/load_to_bigquery/	Scripts for raw file ingestion into BigQuery
dbt_warehouse/models/staging/	Standardised stg_* source models
dbt_warehouse/models/intermediate/	Reusable int_* derived logic
dbt_warehouse/models/marts/	Business-ready fact, dimension, and mart models
dbt_warehouse/models/monitoring/	Query cost and freshness monitoring models
dbt_warehouse/tests/	dbt schema and custom data quality tests
dbt_warehouse/snapshots/	SCD2 / historical snapshots
dbt_warehouse/seeds/	Static seed files (e.g. category mappings)
dbt_warehouse/macros/	Reusable Jinja macros
airflow/dags/	Orchestration DAG definitions
gx/	Great Expectations configuration and checkpoints
dashboards/	Dashboard screenshots and published links
sql_analysis/	8–12 standalone analytical SQL files for portfolio
docs/	Architecture diagram, data model diagram, write-ups
.github/workflows/	GitHub Actions CI pipeline
docker-compose.yml	Local environment configuration
README.md	Project overview and setup instructions

Phase 2 — Raw Ingestion Layer

Load every source file exactly as-is into BigQuery raw tables. Do not apply any business logic at this layer.

Raw Tables

- raw_orders
- raw_order_items
- raw_customers
- raw_products
- raw_sellers
- raw_payments
- raw_reviews
- raw_geolocation
- raw_product_category_translation

Ingestion Rules

- Keep raw tables immutable — never overwrite
- Add an ingested_at timestamp to every row
- Store source filename and batch ID
- Do not apply cleaning or business logic here

Deliverables

- Data dictionary
- Row counts per table
- Null profile
- Duplicate profile
- Source-to-table mapping document

Phase 3 — Staging Layer in dbt

Create stg_* models that standardise and clean raw data. Good staging work demonstrates warehouse hygiene and maturity to recruiters.

Staging Model Responsibilities

- Standardise column names
- Cast types correctly
- Normalise timestamps and dates
- Trim strings
- Handle obvious null / blank issues
- Deduplicate records

- Create surrogate keys where useful

Staging Models to Build

- stg_orders
- stg_order_items
- stg_customers
- stg_payments
- stg_reviews
- stg_products
- stg_sellers
- stg_geolocation

Phase 4 — Intermediate Models

Build reusable logic that multiple downstream models can reference. Derive all calculated fields here, not in the marts.

Intermediate Models

- int_order_value_by_order
- int_payment_summary_by_order
- int_delivery_timeline
- int_customer_first_order
- int_seller_delivery_performance
- int_order_item_pairs (basket analysis)
- int_order_geo_points
- int_review_delivery_join

Important Derived Fields

Category	Derived Fields
Dates	Purchase date, approved date, delivered date, order cycle time
Delivery	Promised vs delivered delay days
Order Metrics	Order value, item count, seller count per order
Payments	Payment installment flags
Customer	First order / repeat order flag, order age buckets
Geography	Customer and seller geography keys

Phase 5 — Core Warehouse Marts

Build business-ready facts, dimensions, and business marts. These are the final outputs consumed by dashboards and analysts.

Dimensions

- dim_customers
- dim_products
- dim_sellers
- dim_dates
- dim_geography

Facts

- fct_orders (order grain)
- fct_order_items (order-item grain)
- fct_payments (payment grain or order-payment grain)
- fct_reviews (review grain)
- fct_delivery_events (order grain)

Business Marts

- mart_exec_daily
- mart_customer_retention
- mart_seller_sla
- mart_category_performance
- mart_payment_behavior
- mart_geo_delivery
- mart_market_basket

SQL Techniques — What Makes This Industry-Level

Deliberately incorporate these techniques throughout your models. They demonstrate real analytical depth and production awareness.

1 Window Functions

Use window functions to avoid costly self-joins and produce richer row-level metrics:

- First / last order per customer
- Rolling 7 / 30 / 90-day GMV
- Seller ranking within category and state
- Cohort retention analysis
- Lag between customer orders
- Percent-of-total contribution

2 Incremental Models

Use dbt incremental models for large fact tables to avoid full rescans:

- fct_orders, fct_order_items, fct_reviews
- Use an ingested_at or batch watermark so only new or changed rows are processed

3 MERGE Upserts

Use BigQuery MERGE when loading curated tables from landing/raw batches into warehouse targets. MERGE is an atomic statement that combines insert, update, and delete logic.

4 Partitioning & Clustering

- Partition big tables by purchase date or delivery date
- Cluster by customer_id, seller_id, product_category, order_status

BigQuery recommends partitioning for date/range filters and clustering for high-cardinality column filters. Both reduce cost and improve performance.

5 Query Observability

Create a monitoring model that reads INFORMATION_SCHEMA.JOBS to track query cost and performance — a strong industry signal showing cost awareness:

- Bytes processed
- Runtime
- User / job patterns
- Most expensive queries

6 Geospatial SQL

Use the Olist location data with BigQuery's GEOGRAPHY and ST_* functions:

- Customer-to-seller distance estimates
- Average delay by distance band
- Geographic delivery hotspots
- Seller coverage maps

7 Cohort & Retention Analysis

- Monthly acquisition cohorts
- Repeat order curves
- Time-to-second-order
- Retention by first category or first seller region

8 Basket Analysis in SQL

- Frequently bought together pairs (self-join on order items)
- Lift-style pair counts

- Category affinity

Data Quality & Governance Layer

dbt Tests

- not_null
- unique
- accepted_values
- relationships
- Custom tests for business rules

Example Business Rule Tests

- Every order in facts must exist in dim_customers
- Delivered date cannot be before purchase date
- Review score must be between 1 and 5
- Payment value must be non-negative
- Approved date must exist for successfully processed orders

Great Expectations

Use Great Expectations on top of dbt tests for richer validation reports and Data Docs — human-readable documentation of expectations and validation results.

Advanced Touch

Publish a data_quality_summary mart with test pass/fail counts by pipeline run.

Source Freshness

Use dbt source freshness on raw tables to simulate source SLAs and monitor whether data is arriving on schedule.

Orchestration Plan — Airflow DAG

Create one Airflow DAG named olist_daily_pipeline. Airflow demonstrates you can move beyond notebooks into scheduled pipelines with logs, reruns, and backfills.

Step	Task
1	Ingest raw files / detect new batch
2	Load raw tables to BigQuery
3	Run lightweight row-count checks

4	Trigger dbt source freshness
5	Run dbt build
6	Run Great Expectations checkpoint
7	Generate dbt docs and quality docs
8	Refresh BI extracts or publish completion status

Dashboard Plan

Build in Looker Studio (free, shareable, native BigQuery connector) and/or Power BI (strong resume appeal). Target four dashboard pages:

Page 1 — Executive Overview

- GMV / revenue proxy
- Total orders
- Average order value (AOV)
- Delivered vs cancelled orders
- Monthly trend
- Top categories
- Top seller regions

Page 2 — Customer Insights

- New vs repeat customers
- Retention matrix
- Time to second order
- Top repeat categories
- Review score by cohort

Page 3 — Operations & Seller Performance

- Average delivery days
- Late delivery rate
- Seller SLA scorecard
- On-time delivery by seller and state
- Review score vs delay

Page 4 — Geographic Logistics

- Customer and seller density map
- Average delay by region
- Distance band vs review score
- High-delay corridors

Optional — SQL-Only ML with BigQuery ML

Add one BigQuery ML model to modernise the project while keeping it SQL-first. Use CREATE MODEL, ML.PREDICT, and ML.EVALUATE — all in SQL.

- Predict late delivery
- Predict low review score
- Predict repeat customer likelihood

Final Portfolio Deliverables

Artifact	Notes
GitHub repo	Full project with all phases committed
README.md	Architecture diagram, setup instructions
Data model diagram	Entity-relationship diagram of warehouse
dbt docs screenshots	Lineage graph, test results
Dashboard screenshots	All 4 pages from Looker Studio or Power BI
sql_analysis/ folder	8–12 polished standalone SQL files
Airflow DAG screenshot	Shows the pipeline steps and runs
Data quality report	Great Expectations validation output
Project write-up	Business problem → data model → insights → tradeoffs
Loom walkthrough (optional)	Short video explaining architecture and findings

Suggested 6-Week Build Order

Week	Focus
Week 1	Repo setup, BigQuery project, raw ingestion, data dictionary
Week 2	Staging models, dbt tests, documentation
Week 3	Intermediate models, fact tables, dimension tables
Week 4	Business marts, window-function analyses, geospatial layer
Week 5	Airflow DAG, Great Expectations, GitHub Actions CI
Week 6	Dashboards, README polish, screenshots, optional BigQuery ML

What Not to Overclaim

Be Honest in Your Portfolio Write-Up

- ✕ Do not claim true marketing attribution from Olist alone
- ✕ Do not claim gross margin unless you add cost data
- ✕ Do not present GA4 sample data as real production data
- ✕ Do not claim SCD2 history unless you actually built a change-feed simulation or mutable dimension source